Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3638562

PDF Download
3638562.pdf
26 January 2026
Total Citations: 9
Total Downloads: 773

Published: 13 February 2024
Online AM: 26 December 2023
Accepted: 15 December 2023
Received: 27 June 2023

Citation in BibTeX format

RESEARCH-ARTICLE

Learning Global and Multi-granularity Local Representation with MLP for Sequential Recommendation

CHAO LONG, Soochow University, Suzhou, Jiangsu, China

HUANHUAN YUAN, Soochow University, Suzhou, Jiangsu, China

JUNHUA FANG, Soochow University, Suzhou, Jiangsu, China

XUEFENG XIAN, Suzhou Vocational University, Suzhou, Jiangsu, China

GUANFENG LIU, Macquarie University, Sydney, NSW, Australia

VICTOR S SHENG SHENG, Texas Tech University, Lubbock, TX, United States

View all

Open Access Support provided by:

Soochow University

Macquarie University

Suzhou Vocational University

Texas Tech University

ACM Transactions on Knowledge Discovery from Data, Volume 18, Issue 4 (May 2024)
https://doi.org/10.1145/3638562
EISSN: 1556-472X

Learning Global and Multi-granularity Local Representation with MLP for Sequential Recommendation

CHAO LONG, HUANHUAN YUAN, and JUNHUA FANG, Soochow University, China
XUEFENG XIAN, Suzhou Vocational University, China
GUANFENG LIU, Macquarie University, Australia
VICTOR S. SHENG, Texas Tech University, USA
PENGPENG ZHAO, Soochow University, China

Sequential recommendation aims to predict the next item of interest to users based on their historical behavior data. Usually, users' global and local preferences jointly affect the final recommendation result in different ways. Most existing works use transformers to globally model sequences, which makes them face the dilemma of quadratic computational complexity when dealing with long sequences. Moreover, the scope setting of the user's local preference is usually static and single, and cannot cover richer multi-level local semantics. To this end, we proposed a parallel architecture for capturing global representation and **M**ulti-granularity **L**ocal dependencies with **M**LP for sequential **Rec**ommendation (**MLM4Rec**). For global representation, we utilize modified MLP-Mixer to capture global information of user sequences due to its simplicity and efficiency. For local representation, we incorporate convolution into MLP and propose a multi-granularity local awareness mechanism for capturing richer local semantic information. Moreover, we introduced a weight pooling method to adaptively fuse local-global representations instead of directly concatenation. Our model has the advantages of low complexity and high efficiency thanks to its simple MLP structure. Experimental results on three public datasets demonstrate the effectiveness of our proposed model. Our code is available here[1].

CCS Concepts: • **Information systems → Recommender systems;**

Additional Key Words and Phrases: Sequential recommendation, global and local, multi-granularity, MLP

[1]https://anonymous.4open.science/r/MLM4Rec

This research was partially supported by the NSFC (62376180, 62176175), the major project of natural science research in Universities of Jiangsu Province (21KJA520004), Suzhou Science and Technology Development Program (SYG202328), the Priority Academic Program Development of Jiangsu Higher Education Institutions and the Exploratory Self-selected Project of the State Key Laboratory of Software Development Environment.
Authors' addresses: C. Long, H. Yuan, J. Fang, and P. Zhao (Corresponding author), School of Computer Science and Technology, Soochow University, No. 1 Shizi Street, Suzhou, China, 215000; e-mails: 20215227009@stu.suda.edu.cn, hhyuan@stu.suda.edu.cn, jhfang@suda.edu.cn, ppzhao@suda.edu.cn; X. Xian (Corresponding author), Suzhou Vocational University, Suzhou, China; e-mail: xianxuefeng@jssvc.edu.cn; G. Liu, Macquarie University, Sydney, Australia; e-mail: guanfeng.liu@mq.edu.au; V. S. Sheng, Texas Tech University, Lubbock, USA; e-mail: Victor.Sheng@ttu.edu.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
1556-4681/2024/02-ART87 $15.00
https://doi.org/10.1145/3638562

ACM Reference format:
Chao Long, Huanhuan Yuan, Junhua Fang, Xuefeng Xian, Guanfeng Liu, Victor S. Sheng, and Pengpeng Zhao. 2024. Learning Global and Multi-granularity Local Representation with MLP for Sequential Recommendation. *ACM Trans. Knowl. Discov. Data.* 18, 4, Article 87 (February 2024), 15 pages.
https://doi.org/10.1145/3638562

1 INTRODUCTION

Recommender systems [14, 28] are widely applied in major e-commerce (TMall, Amazon) and streaming media (Netflix) platforms because they can provide users with more convenient and efficient personalized services. Early collaborative filtering algorithms used matrix factorization to model users' static interests. However, users' interests are not always static, so sequential recommendation that can capture dynamic interest transition patterns has gradually become a research hotspot.

Existing mainstream sequential recommendation models, such as Caser [19] usually focus on the user's local preference while GRU4Rec [4] and SASRec [7] tend to capture global interest in the interaction sequence. However, users' interests are often complex and changeable and are affected by many factors in real life. Therefore, considering both local preferences and global interests will help to produce more comprehensive and accurate prediction results. Some works [2, 6] have been devoted to better integrating local and global information for sequential recommendation. But they usually default to using transformers to globally model sequences, which has been shown to suffer from secondary computational complexity drawbacks when dealing with long sequences. Therefore, it is necessary to design a simple and efficient architecture for capturing global sequence representation.

At the same time, existing work rarely considers whether the range size of the user's local dependency is appropriate, which is crucial for capturing the best user's local preference. We consider Tom's example in Figure 1, when Tom's local preference unit size is 1, we can recommend the item "film" to him according to the local content "camera". If the size of the preference unit is increased to 3, the local contexts "suitcase" and "briefcase" indicate that the user's current preference is to travel, so "bag" may be a better suggestion. In addition, if we continue to increase the size of the preference unit to 5, we find that, based on the two newly added contextual items "backpacks" and "tent", we can speculate that Tom is likely to go mountaineering camping, so a pair of hiking boots might be a more appropriate recommendation. The above examples show that different local preference unit sizes may have different effects on the final recommendation results, and comprehensively considering multi-granularity local information will produce more comprehensive and richer recommendation results. However, how to model users' multi-granularity local preferences and fuse them adaptively is still an open question.

In order to solve the two problems mentioned above, we propose an efficient architecture learning **Multi-granularity Local information with MLP for sequential Recommendation (MLM4Rec)**. Unlike previous work that used standard convolutions to capture local information, our **Convolution Local Enhancement (CLE)** module applies depth-wise convolutions to each item and its neighbors in the sequence to obtain local enhancement weights. We then use this weight to enhance the local representation of each item through a linear gating mechanism. In order to learn higher-quality local representations, we perform a fine-to-coarse multi-granularity division in sequence to capture local weight at different granularities. Correspondingly, we extend the original single-granularity gating into a multi-level gating mechanism to adaptively fuse multi-granularity local information. At the same time, we use MLP-Mixer [21] to model the global dependencies of user interaction sequence, which has the advantage of maintaining low linear complexity while achieving comparable experimental results to Transformer-based models.

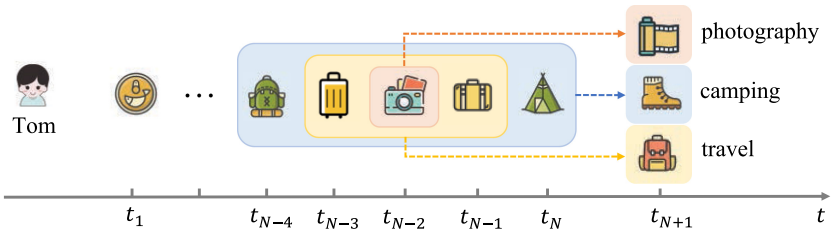


Fig. 1. An illustration of multi-granularity local information awareness mechanism for sequential recommendation.

Furthermore, we design an **Adaptive Fusion Layer (AFL)** to fuse multi-granularity local and global preferences for generating a richer and more comprehensive representation of user preferences. It should be noted that our model adopts MLP as the backbone architecture to capture local and global sequential transformation patterns, giving it an advantage in linear complexity compared to Transformer-based sequential recommendation models.

In summary, the contributions of our paper are as follows:

- We propose an efficient MLP architecture to capture local dependencies and global representation in sequential recommendation while maintaining low computation complexity. Furthermore, we design an adaptive fusion layer to enable local and global information to communicate with each other.
- We innovatively propose a **Convolution Local Enhancement (CLE)** MLP to capture local preference representation and introduce multi-granularity local awareness mechanism for richer local semantic information.
- Through extensive experiments on three datasets, we observe that our proposed method can significantly outperform the state-of-the-art methods, thereby validating the effectiveness of our method.

2 RELATED WORK

2.1 Sequential Recommendation

Early solutions on sequential recommendation were mainly based on Markov chains [16] which calculate an item-item transition probability matrix to predict the next item. Recently, with the rapid development of deep learning, CNN-based [19, 27] models and RNN-based [4] models have achieved remarkable results. With the great success of transformers [23] in the NLP field, researchers are also working hard to explore the potential of applying them to sequential recommendation tasks. The most successful ones are SASRec [7] and BERT4Rec [18]. Both of them use the self-attention mechanism to calculate the attention weight between each item pair. Recent advances in MLP-based architectures [12, 20–22, 26] argue that MLP can learn vision representation by simply stacking multiple MLP layers while maintaining competitive performance. MLP4Rec [11] first investigates the possibility of using MLP architecture as a substitute for Transformer-based methods in sequential recommendation. It expands the MLP-Mixer from the original two-dimensional to three-dimensional and introduces the attribute information of items, so that it can simultaneously model the sequence-dependent associations across locations, channels, and features. In this paper, we study the potential of capturing local and global dependencies in sequential recommendation with MLP meanwhile maintaining low complexity.

2.2 Local Global Awareness

Local global information awareness is a topic that has received much attention in the recommender system scenario. [25] proposed a recurrent convolutional network, which uses RNN to learn

complex global sequence dependencies and uses CNN to extract local dependencies, for merging global sequence representation with local dependencies. [13] combined local generative models that capture different user preferences for global prediction. [2] uses a parametric adaptive predictor to generate a more flexible masking score that places local constraints on the global sequence. AdaMCT [6] adaptively incorporates local inductive bias into global sequence representation by generating a learnable importance coefficient. However, most existing local representation learning methods only focus on how to design more complex and efficient models, but ignore the adaptive setting of the window size for local preference learning. The difference is that our model designs a multi-granularity local enhancement mechanism to capture richer and more comprehensive local dependency representations.

2.3 MLP-based Architecture

Recent advances in MLP-based architectures [12, 20–22, 26] argue that convolutional and attention blocks in networks are unnecessary, and MLP can replace all of them to learn vision representation by simply stacking multiple MLP layers while maintaining linear time complexity. Among them, MLP-Mixer [21] is usually considered as the most representative work. By using separate MLP blocks to learn the cross-location correlations and cross-channel correlations from images, MLP-Mixer shows comparable performance against Transformer-based and CNN-based methods. Given the advantages of MLP-based models, there have been works applying MLP-only architecture to recommender systems. MOI-Mixer [10] first investigates the possibility of using MLP architecture as a substitute for Transformer-based methods in sequential recommendation. They proposed a **Multi-Order Interaction layer (MOI-Mixer)** to capture higher-order sequential dependencies [24], and MOI-Mixer still shows subpar performance compared with Transformer-based methods. Beyond that, MLP4Rec [15] models usersâĂŹ dynamic preferences by extending MLP-Mixer from 2D to 3D, and introduced attribute information while maintaining competitive performance. Different from previous static architecture, our method dynamically models user interaction sequential patterns and can surpass the performance of transformer.

3 OUR PROPOSED MODEL

The overall architecture of MLM4Rec is presented in Figure 2. Details of each module are elaborated on below.

3.1 Problem Definition

Assume that we have a set of users and items, denoted by $\mathcal{U}$ and $\mathcal{V}$, respectively, where $u \in \mathcal{U}$ denotes a user and $v \in \mathcal{V}$ denotes an item. The numbers of users and items are denoted as $|\mathcal{U}|$ and $|\mathcal{V}|$, respectively. Generally, a user u has a chronologically-ordered interaction sequence with items $S^u = \{v_1^u, v_2^u, \ldots, v_{|S^u|}^u\}$, where $|S^u|$ is the number of interactions and v_t^u is the t-th item that the user u has interacted with.

For the purpose of modeling, we establish a fixed sequence length N to standardize the input data size. Sequences longer than N are truncated to the nearest N interactions, constructing the training data. Conversely, sequences shorter than N are padded at the sequence head with the 'pad' item, ensuring a uniform length of N.

The primary objective in sequential recommendation tasks is to predict the $N+1$-th item based on the last N interaction sequences of user u. To achieve this, our approach incorporates both global and local modeling processes. The global modeling phase encompasses learning overarching user preferences and item characteristics from the entire interaction history, while the local modeling phase focuses on capturing immediate sequential patterns within the last N interactions to enhance recommendation accuracy.

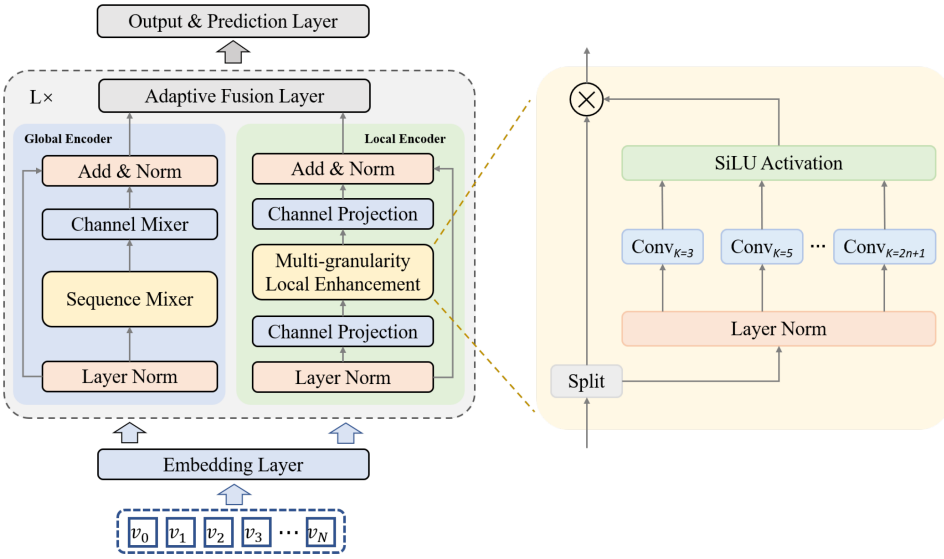


Fig. 2. Overview of our proposed model MLM4Rec. The main body consists of a Global Encoder and a Local Encoder which are processed in parallel. The details of key component named Multi-granularity Local Enhancement in the Local Encoder are shown on the right.

3.2 Overall Architecture

Now we introduce the overview of MLM4Rec - a sequential recommender system based solely on MLP architectures, which can efficiently capture global representations and local dependencies in the input sequence and fuse them together adaptively. The overall architecture of our model is presented in Figure 2.

The embedding layer takes a sequence of n number of items as an input, and each item is projected into a d-dimensional vector resulting in an input matrix of $X \in \mathbb{R}^{n \times d}$. Then, the embedded input X is fed into the Local Encoder and Global Encoder, each of which is made up of several MLP-based networks for mixing spatial-channel information. For Local Encoder, we employ our proposed **convolutional gated MLP (cgMLP)** to consider the local representation dependencies of each item in the sequence, and use a gating mechanism to make local-aware adaptive updates to the original representation. Furthermore, we innovatively propose a multi-granularity local perception mechanism and fuse it with cgMLP to capture richer local information. For Global Encoder, we utilize a simple MLP-Mixer for capturing global dependencies. Finally, the separated local representation after the Local Encoder and the global representation from Global Encoder will be fused adaptively by Adaptive Fusion Layer, and then feed into the next layer. Once the input undergoes the Local Global Encoder module, the encoded representation is passed to the Prediction Layer for the final next item prediction problem. The Prediction Layer is composed of a two-layer **Feed-Forward Network (FFN)** with GELU activation [3], followed by the softmax function as done in [18].

3.3 Embedding Layer

Following the commonly used strategy in sequential recommender systems [7, 18, 29], we use a lookup table to transform the item ID into embedding vectors. The embedding technique projects a high-dimensional one-hot encoding into a low-dimensional dense vector by maintaining an

embedding matrix $E \in \mathbb{R}^{|\mathcal{I}| \times D}$, where D is the embedding size. Then given user u's interaction sequence $S^u = \{v_1^u, v_2^u, \ldots, v_N^u\}$, we can obtain the corresponding embedding matrix $\mathbf{e}^u \in \mathbb{R}^{N \times D}$ by indexing in this matrix using item id.

3.4 Global Encoder

Our Global Encoder is mainly composed of two components, Sequence Mixer and Channel Mixer, which are essentially two MLPs with activation functions to model the relationship between the sequence and channel dimensions of the input embedding matrix, respectively.

3.4.1 Sequence Mixer. We first apply the sequence mixer to the input representation to fuse the cross-item information within the sequence to each item's embedding vector. Given the sequence representation of user u, $\mathbf{e}^u \in \mathbb{R}^{N \times D}$. We define the output of Sequence Mixer at each layer as follows:

$$\mathbf{y}_s = \mathbf{e}^u + \mathbf{W}_2(\text{GELU}(\mathbf{W}_1 \text{LayerNorm}(\mathbf{e}^u) + \mathbf{b}_1)) + \mathbf{b}_2 \tag{1}$$

where $\mathbf{y}_s \in \mathbb{R}^{N \times D}$ is output of the Sequence Mixer that contains cross-item correlations, $\mathbf{W}_1 \in \mathbb{R}^{H_s \times N}, \mathbf{W}_2 \in \mathbb{R}^{N \times H_s}$ signify the learnable weights representing the first and second fully connected layer in the Sequence Mixer, H_s is the tunable dimension of hidden layer, and $\mathbf{b}_1 \in \mathbb{R}^{H_s}, \mathbf{b}_2 \in \mathbb{R}^N$ denote the bias vectors. Here we use **Gaussian Error Linear Unit (GELU)** as our activation function. We employ layer normalization and residual connection to maintain the stability of the training process. In addition, to prevent overfitting, we also use a dropout mechanism after each sequence mixer to make the model converge faster.

Note that while previous architectures with self-attention layers utilize positional embedding to provide either absolute or relative positional information in a given sequence to cope with the permutation invariance property [7, 18], sequence mixer does not require the positional embedding since the sequence mixing MLP is already sensitive to the order of the tokens, allowing inherent learning of position of each sequence.

3.4.2 Channel Mixer. Sharing a similar macro-structure to the Sequence Mixer, the objective of the Channel Mixer is to learn channel correlation within each itemâĂŹs embedding vector. The only difference is that the input dimension of the channel mixer is the size of the embedding representation rather than the sequence length. Given the sequential representation $\mathbf{y}_s$ processed by sequence mixer. We denote the computation of the channel mixer output as follows:

$$\mathbf{Y}_g = \mathbf{y}_s + \text{GELU}(\text{LayerNorm}(\mathbf{y}_s)\mathbf{W}_3 + \mathbf{b}_3)\mathbf{W}_4 + \mathbf{b}_4 \tag{2}$$

where $\mathbf{Y}_g \in \mathbb{R}^{N \times D}$ is the output of the Channel Mixer layer that contains cross-channel correlations, $\mathbf{W}_3 \in \mathbb{R}^{D \times H_c}, \mathbf{W}_4 \in \mathbb{R}^{H_c \times D}$ denote the learnable weights in the Channel Mixer, H_c is the tunable hidden size, and $\mathbf{b}_3 \in \mathbb{R}^{H_c}, \mathbf{b}_4 \in \mathbb{R}^D$ denote the bias vectors. Similarly to the Sequence Mixer, we employ layer normalization, residual connection and dropout methods for the same purpose.

3.5 Local Encoder

Now we introduce two core components of our proposed local encoder: local dependency enhancement module and multi-granularity local information awareness mechanism.

3.5.1 Local Dependency Enhancement. This module consists of a channel projection, an activation function Gaussian Error Linear Unit (GELU), a Convolutional Local Enhancement (CLE) and another channel projection. Additionally, the initial layer norm and the final dropout modules are employed for the purpose of training stability and preventing overfitting. Specifically, given user

u's input sequence representation $\mathbf{e}^u$, the output is calculated as follows:

$$\mathbf{H} = \text{GELU}(\text{LayerNorm}(\mathbf{e}^u)\mathbf{W}_5 + \mathbf{b}_5) \tag{3}$$

$$\mathbf{Y}_l = \mathbf{e}^u + \text{CLE}(\mathbf{H})\mathbf{W}_6 + \mathbf{b}_6 \tag{4}$$

where $\mathbf{W}_5 \in \mathbb{R}^{D\times H_l}$, $\mathbf{W}_6 \in \mathbb{R}^{H_l/2\times D}$ denote the two channel projections and $\mathbf{b}_5 \in \mathbb{R}^{H_l}, \mathbf{b}_6 \in \mathbb{R}^D$ denote bias vectors. H_l is the hidden dimension for capturing richer latent semantics. Here we set $H_l = 4D$ for convenience.

The core idea of the CLE module is to enhance each item's representation by aggregating neighbor information. We first split the input sequence representation $\mathbf{H} \in \mathbb{R}^{N\times H_l}$ equally along the embedding dimension, resulting in two new sequences representation $\mathbf{H}_1, \mathbf{H}_2 \in \mathbb{R}^{N\times H_l/2}$. Then for the purpose of local dependency capture, $\mathbf{H}_2$ is normalized and processed by a depth-wise convolution along the time dimension. Specifically, the calculation process of the final output of the CLE is as follows:

$$\text{CLE}(\mathbf{H}) = \mathbf{H}_1 \odot \mathbf{H}_2' \tag{5}$$

$$\mathbf{H}_2' = \text{SiLU}(\text{DWConv}(\text{LayerNorm}(\mathbf{H}_2))) \tag{6}$$

where $\odot$ denotes element-wise product, DWConv denotes the depth-wise convolution [5] and $\mathbf{H}_2' \in \mathbb{R}^{N\times H_l/2}$ represents the convolution local enhancement weight. Here we utilize the **Sigmoid Linear Unit (SiLU)** function as our activation function. Through the Hadamard product of $\mathbf{H}_1$ and $\mathbf{H}_2'$, we can get the sequence representation after local enhancement.

Depth-wise convolution (DWConv) is a specific convolutional operation within convolutional neural networks that performs spatial convolution on each input channel separately. Its advantages lie in reducing parameters and computational complexity while maintaining model representational capacity. Additionally, the utilization of SiLU and GELU activation functions stems from their nonlinear characteristics, contributing to improved model fitting and expressive capabilities.

3.5.2 Multi-granularity Local Awareness. Depth-wise convolution plays a crucial role in capturing local information, and the question of how much the convolution kernel size K should be set is still open. Previous works usually set it directly as a hyper-parameter for manual tuning, which is considered to be lacking in adaptiveness. Therefore, we propose a multi-granularity local information awareness mechanism, which considers different local preferences at multiple scales when modeling local dependencies, thereby providing more rich and comprehensive neighbor information guidance for pivot items.

Specifically, we set different sizes of convolution kernels $K_1, K_2, \ldots, K_n$ ($K_1 = 3, K_2 = 5, \ldots, K_n = 2n + 1$) to capture the local information in the first-order neighbors, the second-order neighbors,..., the n-order neighbors, respectively. And then the element-wise product is used to fuse the local information of the n granularities, so that the obtained local convolution gating weights can contain a wider and more comprehensive range of local information. Reflected in the formula, we rewrite the calculation process of $\mathbf{H}_2'$ in Equation (6) as follows:

$$\mathbf{H}_2' = \prod_{k=1}^{n} \text{SiLU}(\text{DWConv}_{2k+1}(\text{LayerNorm}(\mathbf{H}_2))) \tag{7}$$

where $2k+1$ denotes the kernel size of depth-wise convolution, n is the largest local scale which is usually no more than half of the average sequence length N. It is worth mentioning that here we use element-wise product instead of direct addition to fuse multi-granularity local weights, which can effectively prevent the unbalanced effects of weight values at different granularities.

3.6 Adaptive Fusion Layer

After we obtain the global representation $\mathbf{Y}_g \in \mathbb{R}^{N\times D}$ and local representation $\mathbf{Y}_l \in \mathbb{R}^{N\times D}$, we need to design an adaptive method to fuse both of them. Considering the feature misalignment between local and global features, the **Adaptive Fusion Layer (AFL)** is designed as the bridge. In order to fuse the two style features, AFL leverages attention-based pooling to summarize the output sequence of each branch into a single vector:

$$\mathbf{y}_g = \text{softmax}(\mathbf{Y}_g W_{att})\mathbf{Y}_g \tag{8}$$

$$\mathbf{y}_l = \text{softmax}(\mathbf{Y}_l W_{att})\mathbf{Y}_l \tag{9}$$

where $\mathbf{y}_g, \mathbf{y}_l \in \mathbb{R}^D$ are global and local pooling representations and $W_{att} \in \mathbb{R}^{D\times 1}$ denotes learnable weight for generating attention scores. Then, the two vectors are projected to two scalars and normalized using softmax to get branch weights:

$$w_g, w_l = \text{softmax}(\mathbf{W}_g\mathbf{y}_g, \mathbf{W}_l\mathbf{y}_l) \tag{10}$$

where $\mathbf{W}_g, \mathbf{W}_l \in \mathbb{R}^{1\times D}$ represent linear transforms. For the merging part, though concatenation is simple and effective, it is difficult to interpret and modify. To mitigate this issue, weighted average is utilized in our work to indicate how global and local relationships are affected in different layers, where the weights are dynamically generated by the model. The final representation $\mathbf{Y}$ that captures both global and local dependencies is calculated as follows:

$$\mathbf{Y} = w_g\mathbf{Y}_g + w_l\mathbf{Y}_l \tag{11}$$

where $\mathbf{Y} \in \mathbb{R}^{N\times D}$.

Since Local Encoder and Global Encoder tend to capture features of different levels (e.g., local vs. global), AFL is utilized to consecutively eliminate the semantic divergence between them, in an interactive fashion. Such a fusion procedure can greatly enhance the global perception capability of local features and the local details of global representations.

3.7 Training and Loss

Following the common setting in sequential recommendation, our training loss is defined by a binary cross entropy loss function as follows:

$$\mathcal{L} = -\log\left(\sigma\left(\mathbf{Y}_N \cdot \mathbf{e}^u_{N+1}\right)\right) - \log\left(1 - \sigma\left(\mathbf{Y}_N \cdot \mathbf{e}^-_{N+1}\right)\right) \tag{12}$$

where σ denotes the sigmoid function, $\mathbf{e}^u_{N+1}$ denotes the ground-truth item at time step N+1, and $\mathbf{e}^-_{N+1}$ is a random sampled negative item. We specify the representation of the last item $\mathbf{Y}_N$ as the final prediction result.

Recently, research advances in MLP-Mixer [12, 20–22, 26] have demonstrated extraordinary competitiveness in various tasks in the field of computer vision, especially in image classification. MLP-Mixer captures cross-location correlation and cross-channel relationships in an image by stacking two MLP layers, token-mixer and channel-mixer. Inspired by this, we introduce MLP-Mixer into the field of recommender systems, modifying the original model appropriately to make it suitable for sequential recommendation tasks. However, the original MLP-Mixer architecture focuses more on how to capture globally effective information, and lacks awareness of the relationship between local representations. Based on this, we improve the linear gating unit into a convolutional gating unit on top of the variant of gMLP to enhance its ability to capture local representations. In addition, we innovatively propose a multi-granularity local information-aware module and insert it seamlessly into cgMLP to adaptively fuse the influence of multi-granularity local neighbors.

Table 1. Statistics of Datasets used in Experiments

DataSets	#users	#items	#actions	avg.length	sparsity
Beauty	22,363	12,101	198,502	8.9	99.73%
Yelp	30,431	20,033	316,354	10.3	99.95%
ML-1M	6,040	3,953	1,000,209	165.6	95.81%

3.8 Complexity Analysis

This section will show the important strength of MLM4Rec, which is the linear computational complexity. The complexity of MLM4Rec is primarily determined by two modules, Global Encoder and Local Encoder. For the Global Encoder, the complexity of sequence mixer is $O(N \times D \times n)$, in which n is the hidden size of sequence mixer, N is the max sequence length, and D is the embedding size. Since the determinant variable here is the maximum sequence length N, we can rewrite that as $O(N)$. Similarly, we have the complexity of the channel mixer as $O(N \times D^2)$, which is also linear with respect to N. For Local Encoder, its time complexity is mainly determined by CLE. The computational costs of the two channel projections are $O(N \times D \times H_l)$ and $O(N \times D \times H_l/2)$, respectively. The depth-wise convolution has $O(N \times K \times H_l/2)$ complexity, where K is the kernel size. The overall complexity is linear with respect to N, although there is a constant factor K. So the final time complexity of our model is $O(N)$, which is linear compared to the time complexity of self-attention $O(N^2 + D^2)$.

4 EXPERIMENTS

In this section, we first describe the datasets, baselines, evaluation metrics and experimental settings. Then we do detailed analysis for the experimental results.

4.1 Datasets

We conduct extensive experiments on three real-world public datasets which are collected from different domains with various sparsity levels.

- Amazon **Beauty** [2]: This dataset contains reviews and ratings of various items that are crawled from Amazon. It is divided into multiple categories, and in this paper we will use "Beauty" category.
- **Yelp** [3]: This dataset is very famous for business recommendation. As it is very large, we follow the common practice and only use the transaction records after January 1st, 2019.
- **MovieLens-1M**: MovieLens is a commonly used benchmark dataset for recommender systems. It contains users' rating history on a wide range of movies with corresponding timestamps. In this study we choose the version of MovieLens-1M (**ML-1M**) [4] for experiments.

Regarding data preprocessing, in order to ensure the quality of the data and make training and evaluation more meaningful, we only use the "5-core" version of the dataset which is the same as previous work [1, 7, 17, 19], that is, to filter out users with less than five items in the historical user interaction sequence, and to remove items with less than five interactions. Table 1 shows the statistics of datasets after preprocessing.

[2] http://jmcauley.ucsd.edu/data/amazon/
[3] https://www.yelp.com/dataset
[4] https://grouplens.org/datasets/movielens/1m/

4.2 Task Settings & Evaluation Metrics

When evaluating our model, we adopt the *leave-one-out* evaluation method commonly used in previous work [1, 7, 19], that is, select the last item as the test data, the penultimate item as the validation data, and the rest of the items as the training set. To avoid biased results [9], we use the entire item set to evaluate all methods and fully sort all items by similarity to generate a recommendation list.

We adopt two most commonly used metrics for performance evaluation, namely Recall and **Normalized Discounted Cumulative Gain (NDCG)**, which are widely used in previous work [17, 29]. Since Recall@1 is equivalent to NDCG@1, we report our results on Recall@5,10,20 and NDCG@5,10,20.

4.3 Baselines

To verify the effectiveness of our method, we compare it with the following representative baselines:

- **GRU4Rec** [4]: It is proposed to use RNN for sequential recommendation, and improved vanilla RNN with **Gated Recurrent Unit (GRU)** and point-wise ranking loss.
- **Caser** [19]: It proposed to model recent actions as an "image" and learn sequential patterns using convolutional filters.
- **SASRec** [7]: It uses a self-attention mechanism in Transformer to model user sequence behavior transition patterns from left to right, which is one of the state-of-the-art baseline methods in sequential recommendation.
- **MLP4Rec** [11]: It applies MLP-Mixer to sequential recommendation and extends it to 3D to fuse item attribute information for more personalized recommendation performance.
- **FMLP-Rec** [30]: It uses an all-MLP model with learnable filter to alleviate the noise problem in sequential recommendation models.
- **Locker** [2]: It introduces local inductive bias into global self-attention to enhance its ability to model short-term interest.

4.4 Implementation Details

All parameters of our baselines are set according to the optimal parameters reported in the original paper. If original papers do not specify preferable hyper-parameters, then we use grid search for hyper-parameter tuning. We use the Adam [8] optimizer to train our model with the learning rate 1e-4, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and l_2 weight decay of 0.01. For key parameters of MLM4Rec, we tune embedding size D, hidden dropout rate p, max kernel size K within {32,64,96,128,256}, $\{0.1, 0.2, \ldots, 0.9\}$, {3,5,7,9}, respectively.

4.5 Overall Performance Comparison

In Table 2, we show the overall comparison of our proposed methods against a wide range of popular baseline methods.

Early deep neural network-based sequential recommendation models such as GRU4Rec and Caser achieved competitive results, suggesting that modeling the local dependencies of sequences is as important as the global representation. SASRec adopts the self-attention mechanism in Transformer for sequence modeling, and achieves better performance compared with GRU4Rec and Caser, which shows that Transformer is more powerful than RNN and CNN in modeling sequential recommendations. MLP4Rec uses MLP-Mixer to capture the three-dimensional correlations of sequence, channel, and item attributes, and can achieve results comparable to Transformer-based models such as SASRec while maintaining linear complexity. FMLP-Rec uses a learnable filter to

Table 2. Performance Comparison of Different Methods on Sequential Recommendation

Database	Model	HR@5	HR@10	HR@20	NDCG@5	NDCG@10	NDCG@20
Beauty	Caser	0.0251	0.0342	0.0643	0.0145	0.0226	0.0298
	GRU4Rec	0.0206	0.0332	0.0526	0.0139	0.0175	0.0221
	SASRec	0.0347	0.0536	0.0833	0.0220	0.0276	0.0355
	MLP4Rec	0.0399	0.0635	0.1006	0.0233	0.0304	0.0403
	FMLP-Rec	0.0370	0.0612	0.0959	0.0241	0.0318	0.0406
	Locker	0.0317	0.0464	0.0674	0.0209	0.0259	0.0311
	MLM4Rec	**0.0524**	**0.0779**	**0.1104**	**0.0344**	**0.0426**	**0.0508**
	Improvement	**31.33%**	**22.68%**	**9.74%**	**42.74%**	**33.96%**	**25.12%**
Yelp	Caser	0.0142	0.0254	0.0406	0.0080	0.0113	0.0179
	GRU4Rec	0.0152	0.0248	0.0371	0.0091	0.0124	0.0145
	SASRec	0.0160	0.0260	0.0443	0.0101	0.0133	0.0179
	MLP4Rec	0.0178	0.0301	0.0518	0.0109	0.0154	0.0197
	FMLP-Rec	0.0166	0.0292	0.0500	0.0102	0.0142	0.0194
	Locker	0.0168	0.0294	0.0506	0.0109	0.0159	0.0209
	MLM4Rec	**0.0197**	**0.0349**	**0.0590**	**0.0124**	**0.0172**	**0.0229**
	Improvement	**10.67%**	**15.95%**	**13.90%**	**13.76%**	**8.18%**	**9.57%**
ML-1M	Caser	0.0912	0.1442	0.2228	0.0565	0.0734	0.0931
	GRU4Rec	0.0993	0.1806	0.2892	0.0574	0.0835	0.1108
	SASRec	0.1108	0.1902	0.3124	0.0648	0.0904	0.1211
	MLP4Rec	0.1431	0.2240	0.3380	0.0976	0.1236	0.1498
	FMLP-Rec	0.1386	0.2180	0.3283	0.0889	0.1146	0.1424
	Locker	0.1617	0.2459	0.3517	0.1085	0.1356	0.1627
	MLM4Rec	**0.1808**	**0.2786**	**0.4038**	**0.1185**	**0.1501**	**0.1816**
	Improvement	**11.81%**	**13.30%**	**14.81%**	**9.22%**	**10.69%**	**11.62%**

Bold scores are the best while underlined scores are the second best.

replace the self-attention layer in the transformer and achieves better performance. It enhances the accuracy of sequential recommendation by attenuating noisy information in the frequency domain. Compared to the previous single perspective of sequence modeling, Locker introduces local inductive bias into self-attention to enhance its ability to model short-term dynamic preferences and outperform SASRec on three datasets.

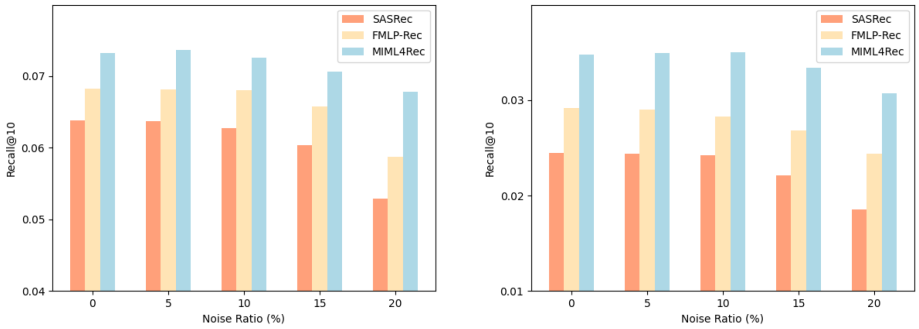
Finally, according to the experimental results, we can clearly see that the performance of our proposed model MLM4Rec can beat all baselines on all datasets and evaluation metrics. Different from these baselines, we used a full MLP parallel architecture to model the local and global patterns of sequence transformation. Thanks to the simplicity of the MLP architecture, our model has low complexity and computational overhead. Meanwhile, the parallel architecture and adaptive fusion module can dynamically adjust the weights of the local and global representations based on the input data.

4.6 Further Analysis

4.6.1 Impact of Kernel Size. The maximum convolution kernel size represents the granularity upper bound in our proposed multi-granularity convolution enhancement mechanism. This enables our local encoder to focus more on local features rather than global features, while avoiding unnecessary noise introduction. We conducted our experiments on three different datasets, Beauty, Yelp, and ML-1M, respectively. The upper bound of the maximum convolution kernel was set to 9. The experimental results are shown in Table 3.

Table 3. Performance with Different Convolution Granularity

Model	Beauty		Yelp		ML-1M	
	HR@20	NDCG@20	HR@20	NDCG@20	HR@20	NDCG@20
K={3}	0.1021	0.0445	0.0543	0.0214	0.3930	0.1803
K={3,5}	0.1082	0.0492	0.0553	0.0220	0.4002	0.1805
K={3,5,7}	**0.1114**	0.0495	**0.0590**	**0.0232**	**0.4038**	**0.1816**
K={3,5,7,9}	0.1104	**0.0508**	0.0555	0.0218	0.3892	0.1772



(a) Toys (b) Yelp

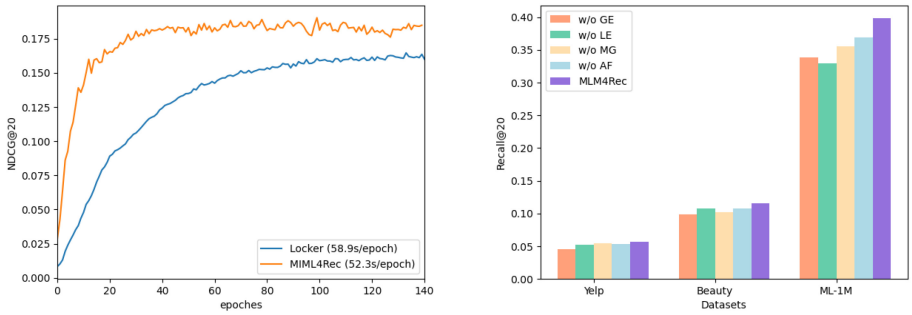
Fig. 3. Performance comparison w.r.t. noise ratio on Toys and Yelp.

From the table we can see that more kernel granularity leads to better results in the Yelp dataset. This shows that considering multi-granularity local dependencies is more helpful for capturing local features than simply using a fixed-size convolution kernel. At the same time, we found that in the Beauty and ML-1M datasets, blindly increasing the size of the convolution kernel does not bring continuous improvement in performance. A possible reason may be that a larger convolution kernel size will increase the risk of introducing non-semantically similar neighbor information, thereby reducing the quality of the captured local user intent representation.

4.6.2 Training Efficiency. To verify the efficiency of our model, we compare the training efficiency of MLM4Rec with Locker on ML-1M. Our experiments mainly focus on two aspects: training speed and convergence speed. To visualize the performance, we recorded the validation results for each epoch and plotted them as a curve in Figure 4(a).

From Figure 4(a), we can observe that MLM4Rec begins to converge around 50 epochs, while Locker's efficiency curve does not begin to flatten until around 80 epochs. In terms of training speed, the training speed of MLM4Rec (52.3s/epoch) is even a little faster than Locker (58.9s/epoch), and at the same time it can achieve better recommendation results. This series of experimental results show that our proposed model is efficient and fast. This is mainly due to the simple structure and low complexity of our designed MLP.

4.6.3 Robustness to Noisy Data. We also conduct experiments on the Toys and Yelp datasets to verify the robustness of MLM4Rec against noisy interactions in the test phase. Specifically, we randomly add a certain proportion (i.e., 5%, 10%, 15%, 20%) of negative items to text sequences. From Figure 3 we can see that adding noisy data deteriorates the performance of SASRec, FMLP-Rec and MLM4Rec. However, the performance drop ratio of MLM4Rec is consistently lower than SASRec and FMLP-Rec, and its performance with 15% noise proportion can still outperform SASRec without noisy dataset on Toys and Yelp. The reason might be the leveraged multi-granularity



(a) Training efficiency (b) Ablation study

Fig. 4. (a) Training efficiency (NDCG@20) on ML-1M dataset including training speed and convergence speed. (b) Performance comparison for ablation study.

local intent unit can denoise the neighbors of each item through a multi-layer convolution gating mechanism, resulting in a more robust local semantic representation.

4.7 Ablation Study

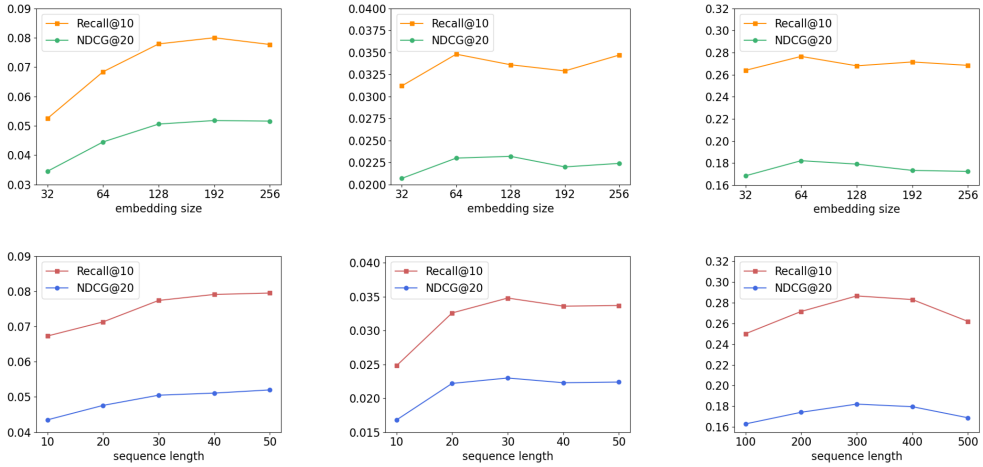
In this section, we perform an ablation study on several key components in our model to analyze the impact of each component on model performance. We conduct experiments on several variants of MLM4Rec by dropping the **Multi-Granularity (MG)** mechanism, **Local Encoder (LE)**, **Global Encoder (GE)**, and **Adaptive Fusion (AF)** layer. The results are shown in Figure 4(b).

First, we remove the most important multi-granularity local awareness mechanism in our model. Experimental results show that the performance of our model drops a lot, which verifies the necessity of considering multiple neighbor information. Then we conduct ablation studies on the two parallel branches of our model, the Local Encoder and Global Encoder, respectively. From Figure 4(b), we can observe that using either of the two for user interest modeling leads to a sharp drop in performance, which illustrates that user intent in real life is affected by both local and global intent. Furthermore, we also found that Global Encoder performs slightly better than Local Encoder on sparse datasets. A possible explanation is that most sequences in sparse datasets are short, and contain limited local intent, so simply modeling the global representation can more accurately capture the real intent of users. While long sequences in ML-1M generally contain more rich local intentions which better reflect current real preferences than global preferences. In addition, we use simple connection or average pooling instead of the adaptive fusion module to fuse the local and global representations. The experimental results show that adaptively adjusting the respective fusion weights according to the local and global representations is the best choice.

4.8 Hyper-parameter Sensitivity

In this section, we mainly investigate the performance sensitivity of our model on three datasets with respect to two hyper-parameters including embedding size D and sequence length N while maintaining other optimal hyper-parameters unchanged. The experiment results are presented in Figure 5.

From the first row, we found that with the increase of the embedding dimension, the performance of the model showed a trend of continuously rising on Beauty. The potential reason may be that the larger the embedding dimension, the stronger the item representation ability. Especially for our multi-granularity local encoder module, since its core component CLE needs to split the



(a) Beauty (b) Yelp (c) ML-1M

Fig. 5. Parameter sensitive w.r.t. embedding size and sequence length on three datasets Beauty, Yelp and ML-1M.

embedding representation, a larger embedding size can provide a richer local representation while avoiding the problem of unnecessary information loss.

From the second row we can find that the maximum sequence length N has a positive correlation with the average maximum length of the dataset. The best performance on Beauty and Yelp is $N = 50$, while on ML-1M, the best performance parameter is $N = 300$. In addition, as N continues to increase, we find that the performance of the model gradually tends to decline on ML-1M, because an excessively large sequence length may introduce more noisy information and interfere with the model's capture of useful information.

5 CONCLUSION

In this paper, we mainly focus on how to design an efficient framework for modeling local dependencies and global information in sequential recommendation. Specifically, we utilize MLP-Mixer to capture global representation, and propose a multi-granularity local enhancement MLP module to capture richer local semantic information. Furthermore, we design an adaptive fusion layer to fuse these two parallel MLP-based branches. Extensive experiments on three datasets show the effectiveness of our model.

REFERENCES

[1] Xiangnan He and Tat-Seng Chua. 2017. Neural factorization machines for sparse predictive analytics. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 355–364.

[2] Zhankui He, Handong Zhao, Zhe Lin, Zhaowen Wang, Ajinkya Kale, and Julian McAuley. 2021. Locker: Locally constrained self-attentive sequential recommendation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 3088–3092.

[3] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415* (2016).

[4] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based recommendations with recurrent neural networks. In *4th International Conference on Learning Representations*.

[5] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).

[6] Juyong Jiang, Jae Boum Kim, Yingtao Luo, Kai Zhang, and Sunghun Kim. 2022. AdaMCT: Adaptive mixture of CNN-transformer for sequential recommendation. *arXiv preprint arXiv:2205.08776* (2022).

[7] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE International Conference on Data Mining*. 197–206.
[8] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*.
[9] Walid Krichene and Steffen Rendle. 2020. On sampled metrics for item recommendation. In *The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1748–1757.
[10] Hojoon Lee, Dongyoon Hwang, Sunghwan Hong, Changyeon Kim, Seungryong Kim, and Jaegul Choo. 2021. MOI-mixer: Improving MLP-mixer with multi order interactions in sequential recommendation. *arXiv preprint arXiv:2108.07505* (2021).
[11] Muyang Li, Xiangyu Zhao, Chuan Lyu, Minghao Zhao, Runze Wu, and Ruocheng Guo. 2022. MLP4Rec: A pure MLP architecture for sequential recommendations. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*. 2138–2144.
[12] Hanxiao Liu, Zihang Dai, David So, and Quoc V. Le. 2021. Pay attention to MLPs. *Advances in Neural Information Processing Systems* 34 (2021), 9204–9215.
[13] Huafeng Liu, Liping Jing, Jingxuan Wen, Zhicheng Wu, Xiaoyi Sun, Jiaqi Wang, Lin Xiao, and Jian Yu. 2020. Deep global and local generative model for recommendation. In *Proceedings of The Web Conference 2020*. 551–561.
[14] Jie Lu, Dianshuang Wu, Mingsong Mao, Wei Wang, and Guangquan Zhang. 2015. Recommender system application developments: A survey. *Decision Support Systems* 74 (2015), 12–32.
[15] Chen Ma, Peng Kang, and Xue Liu. 2019. Hierarchical gating networks for sequential recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 825–833.
[16] Steffen Rendle. 2010. Factorization machines. In *The 10th IEEE International Conference on Data Mining*. 995–1000.
[17] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized Markov chains for next-basket recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. 811–820.
[18] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 1441–1450.
[19] Jiaxi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. 565–573.
[20] Yuki Tatsunami and Masato Taki. 2021. RaftMLP: Do MLP-based models dream of winning over computer vision? *arXiv preprint arXiv:2108.04384* (2021).
[21] Ilya O. Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, Mario Lucic, and Alexey Dosovitskiy. 2021. MLP-mixer: An all-MLP architecture for vision. NeurIPS 2021, 24261–24272.
[22] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, and Hervé Jégou. 2023. ResMLP: Feedforward networks for image classification with data-efficient training. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 4 (2023), 5314–5321.
[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems 30*. 5998–6008.
[24] Shoujin Wang, Liang Hu, Yan Wang, Longbing Cao, Quan Z. Sheng, and Mehmet Orgun. 2019. Sequential recommender systems: Challenges, progress and prospects. *arXiv preprint arXiv:2001.04830* (2019).
[25] Chengfeng Xu, Pengpeng Zhao, Yanchi Liu, Jiajie Xu, Victor S. Sheng, Zhiming Cui, Xiaofang Zhou, and Hui Xiong. 2019. Recurrent convolutional neural network for sequential recommendation. In *The World Wide Web Conference*. 3398–3404.
[26] Tan Yu, Xu Li, Yunfeng Cai, Mingming Sun, and Ping Li. 2022. S2-MLP: Spatial-shift MLP architecture for vision. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 297–306.
[27] Fajie Yuan, Alexandros Karatzoglou, Ioannis Arapakis, Joemon M. Jose, and Xiangnan He. 2019. A simple convolutional generative network for next item recommendation. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. 582–590.
[28] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *Comput. Surveys* 52, 1 (2019), 1–38.
[29] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. 2019. Feature-level deeper self-attention network for sequential recommendation. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*. 4320–4326.
[30] Kun Zhou, Hui Yu, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Filter-enhanced MLP is all you need for sequential recommendation. In *Proceedings of the ACM Web Conference 2022*. 2388–2399.

Received 27 June 2023; accepted 15 December 2023